

Java Backend Architecture

Interview Series

Chapter 9.10

Cache Eviction Policies & Consistency

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 9.10 – Cache Eviction Policies & Consistency

Introduction

Two of the hardest problems in caching are **eviction** and **consistency**. Eviction determines which entries are removed when the cache reaches capacity — choosing poorly means evicting hot entries and tanking your hit rate. Consistency determines how closely the cached data matches the ground truth in the database — choosing incorrectly means serving stale or incorrect data to users. This chapter covers all major eviction algorithms (LRU, LFU, ARC, W-TinyLFU, FIFO), time-based expiry modes (expireAfterWrite, expireAfterAccess, refreshAfterWrite), and cache consistency models with their implementation patterns.

Core Concepts & Definitions

Eviction Policy	Algorithm that selects which cache entry to remove when cache reaches capacity.
LRU	Least Recently Used: evicts the entry that has not been accessed for the longest time.
LFU	Least Frequently Used: evicts the entry with the lowest access frequency count.
ARC	Adaptive Replacement Cache: self-tuning between LRU and LFU using ghost lists and a balance parameter p.
W-TinyLFU	Window + TinyLFU: Caffeine's algorithm; frequency sketch + segmented LRU; near-optimal hit rate.
FIFO	First In First Out: evicts the oldest inserted entry regardless of access pattern.
SLRU	Segmented LRU: hot + warm segments; entry promoted to hot on re-access; simple and effective.
expireAfterWrite	TTL measured from the time the entry was created or last written.
expireAfterAccess	TTL measured from the time the entry was last read or written — hot entries persist.
refreshAfterWrite	Soft TTL: stale entry served while async reload runs in background.
Cache Eviction	Removal of entries to free space, triggered by capacity limit.
Cache Expiration	Removal of entries due to TTL — time-based, not capacity-based.
Strong Consistency	Cache and DB always have identical values; writes update both atomically.
Eventual Consistency	Cache lags DB by at most one TTL period; most common model.
Cache Invalidation	Explicitly removing or marking stale a cache entry after the underlying data changes.
CDC (Change Data Capture)	Database log-based mechanism (Debezium) to detect DB changes and propagate invalidations.

Eviction Policies

Eviction Policy Comparison

LRU

Least Recently Used
Evicts entry not accessed for longest time

Pro: Simple, handles recency

Con: Scan vulnerability

LFU

Least Frequently Used
Evicts entry with lowest access frequency

Pro: Protects hot entries

Con: New entries starved

ARC

Adaptive Replacement Cache
Self-tunes LRU+LFU balance
Ghost lists track history

Pro: Adapts to workload

Con: Patent, more memory

W-TinyLFU (Caffeine)

Freq sketch + window LRU

FIFO

First-in first-out
Simple but

Random

Evict random entry; simple

SLRU (Segmented)

Hot + warm segments

W-TinyLFU (Caffeine) provides near-optimal OPT hit rate across all workload types

TTL and Time-Based Expiry Modes

TTL and Time-Based Expiry Modes

expireAfterWrite

TTL measured from creation time
Suitable for data freshness

expireAfterAccess

TTL measured from last access
Hot entries stay
Cold entries leave

refreshAfterWrite

Soft TTL: stale entry served while async reload runs
Zero-latency refresh

Write → TTL starts here

Access → TTL starts here

Write → TTL starts here

Combine: `expireAfterWrite(10m).refreshAfterWrite(8m)` — refresh at 8m, hard expiry at 10m

Redis: SETEX key ttl value (expireAfterWrite) | EXPIRE key ttl after access (not native)

Cache Consistency Models

Cache Consistency Models

Strong Consistency

Cache + DB always identical. Write to both or invalidate. Highest latency.

Eventual Consistency

Cache lags DB by max one TTL period. Higher performance. Most common.

Session (Read-Your-Writes) Consistency

User always sees their own writes. Route user to same cache node / short TTL.

TTL Expiry

Event-Driven
Invalidation

Write-Through
Evict

Versioned
Keys

CDC
(Debezium)

Invalidation Strategies (L-R: weakest to strongest consistency)

LRU Implementation (LinkedHashMap)

```
// LRU Cache using LinkedHashMap (accessOrder=true)
public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder=true
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

// Thread-safe: wrap with Collections.synchronizedMap()
Map<Long, Product> lruCache = Collections.synchronizedMap(new LRUCache<>(1000));
// Or use Caffeine LRU (automatically uses SLRU/W-TinyLFU internally)
Cache<Long, Product> caffeineCache = Caffeine.newBuilder().maximumSize(1000).build();
```

LFU with Count-Min Sketch (W-TinyLFU Internals)

```
// Count-Min Sketch: space-efficient frequency approximation // Used by Caffeine as TinyLFU admission filter
// Conceptual implementation:
public class CountMinSketch {
    private final int[][] table;
    private final int width, depth;
    private final int[] seeds;
    public CountMinSketch(int width, int depth) {
        this.width = width;
        this.depth = depth;
        this.table = new int[depth][width];
        this.seeds = generateHashSeeds(depth);
    }
    public void increment(long key) {
        for (int i = 0; i < depth; i++) {
            int col = hash(key, seeds[i]) % width;
            table[i][col] = Math.min(table[i][col] + 1, 15); // 4-bit
        }
    }
    public int estimate(long key) {
        int min = Integer.MAX_VALUE;
        for (int i = 0; i < depth; i++) {
            min = Math.min(min, table[i][hash(key, seeds[i]) % width]);
        }
        return min; // minimum = best frequency estimate
    }
}

// Admission: admit new entry only if freq(new) > freq(victim from Probation)
// W-TinyLFU: new entries enter Window Cache (1%), graduates to LFU via sketch
```

ARC (Adaptive Replacement Cache) Algorithm

```
// ARC maintains 4 lists:
// T1: pages seen exactly once recently (recency)
// T2: pages seen more than once recently (frequency)
// B1: ghost entries evicted from T1 (tracks history)
// B2: ghost entries evicted from T2 (tracks history)
// Parameter p: target size for T1, self-adjusted
// On cache hit in T1 or T2: move to T2 (MRU)
// On cache miss:
// If key in B1 (ghost): increase p (favor recency), fetch from DB
// If key in B2 (ghost): decrease p (favor frequency), fetch from DB
// Otherwise: full miss, fetch from DB, add to T1
// ARC self-tunes: workload with temporal locality → large T1
// workload with frequency locality → large T2
// Note: ARC is patented by IBM; Caffeine uses W-TinyLFU instead
// ZFS uses ARC (L1 ARC = RAM, L2 ARC = SSD)
```

Configuring Eviction in Caffeine

```
// Size-based eviction (entry count) Cache<Long, Product> sizeCache = Caffeine.newBuilder()
    .maximumSize(10_000) .build(); // Weight-based eviction (variable size objects) Cache<Long,
    byte[]> weightCache = Caffeine.newBuilder() .maximumWeight(100 * 1024 * 1024) // 100 MB
    .weigher((Long k, byte[] v) -> v.length) .build(); // Time-based eviction Cache<Long, Product>
    timeCache = Caffeine.newBuilder() .expireAfterWrite(Duration.ofMinutes(10))
    .expireAfterAccess(Duration.ofMinutes(5)) .refreshAfterWrite(Duration.ofMinutes(8)) .build(id
    -> repo.findById(id).orElse(null)); // Custom expiry per-entry (variable TTL) Cache<Long,
    Product> customTtlCache = Caffeine.newBuilder() .expireAfter(new Expiry<Long, Product>() {
    public long expireAfterCreate(Long k, Product v, long t) { // VIP products cached longer return
    v.isVip() ? TimeUnit.HOURS.toNanos(1) : TimeUnit.MINUTES.toNanos(10); } public long
    expireAfterUpdate(Long k, Product v, long t, long d) { return d; // no change } public long
    expireAfterRead(Long k, Product v, long t, long d) { return d; // no change } }) .build();
```

Redis Eviction Policies

Policy	Scope	Algorithm	Best For
noeviction	All keys	Return error on OOM	Persistent DB (not cache)
allkeys-lru	All keys	Evict LRU key	General cache workload
allkeys-lfu	All keys	Evict least-frequent key	Skewed access patterns
allkeys-random	All keys	Random eviction	Uniform access patterns
volatile-lru	TTL keys only	Evict LRU with TTL	Mix of persistent + cached
volatile-lfu	TTL keys only	Evict LFU with TTL	TTL-scoped frequency eviction
volatile-random	TTL keys only	Random with TTL	Simple TTL-scoped eviction
volatile-ttl	TTL keys only	Evict soonest-expiring	Minimize stale entries

```
# redis.conf maxmemory 2gb maxmemory-policy allkeys-lfu # recommended for pure cache
maxmemory-samples 10 # LRU/LFU approximation samples (higher=accurate) # Monitor eviction #
redis-cli INFO stats | grep evicted # evicted_keys:12345 – high number means cache too small or
policy misconfigured
```

Cache Invalidation Patterns

```
// 1. TTL-based eventual consistency (simplest) @Cacheable(value = "products", key = "#id")
public Product findById(Long id) { return repo.findById(id).orElseThrow(); } // Staleness
window = TTL duration // 2. Write-through invalidation on mutation @CacheEvict(value =
"products", key = "#product.id") @Transactional public Product update(Product product) { return
repo.save(product); // evict on commit } // 3. Event-driven invalidation via Kafka
@KafkaListener(topics = "product-events") public void onProductEvent(ProductEvent event) { if
(event.getType() == UPDATED || event.getType() == DELETED) { cacheManager.getCache("products")
.evict(event.getProductid()); } } // 4. CDC with Debezium – DB log-based invalidation //
Debezium tails PostgreSQL WAL, publishes RowChangedEvent to Kafka // Consumer evicts cache
entries for changed rows // Most reliable: catches ALL DB changes including direct SQL updates
```

Versioned Cache Keys (Strong Consistency Without Eviction)

```
// Include version in cache key – avoids invalidation entirely // On entity update: version
increments → new cache key → old key auto-expires @Cacheable(value = "products", key = "#id +
:' + #version") public Product findByIdAndVersion(Long id, long version) { return
repo.findById(id).orElseThrow(); } // Or derive version from DB: @Cacheable(value = "products",
```

```
key = "T(String).valueOf(#id) + ':' + #entity.version") public Product cache(Product entity) {
return entity; } // Client always fetches current version from a lightweight metadata endpoint:
// GET /products/42/version → {"version": 17} // Then: GET /products/42?v=17 → cache key
"42:17" // Trade-off: stale keys accumulate until TTL; set TTL >> version change frequency
```

CRDT-Based Cache Consistency (Distributed)

```
// CRDT (Conflict-free Replicated Data Type) for distributed cache consistency // Used in:
Hazelcast Active-Active WAN, Redis Enterprise Active-Active // G-Counter CRDT: increment-only
counter, merges by taking max per node // LWW (Last-Write-Wins) register: conflict resolved by
timestamp // PN-Counter: pair of G-counters (increment + decrement) // Example: Hazelcast
replicated map (CRDT semantics) ReplicatedMap<Long, Product> replicatedMap =
hz.getReplicatedMap("products"); // All cluster members have full copy // Writes broadcast to
all members; eventual consistency via vector clocks // Read: local, zero latency // Write:
async broadcast // Use when: geo-distributed, can tolerate brief inconsistency, // need local
read latency everywhere
```

Monitoring Eviction & Consistency in Production

```
// Caffeine metrics via Micrometer CacheStats stats = cache.stats(); double hitRate =
stats.hitRate(); // target > 0.85 long evictions = stats.evictionCount(); // rising fast =
cache too small long loadTime = stats.totalLoadTime(); // nanoseconds double missRate =
stats.missRate(); // 1 - hitRate // Alert conditions: // eviction_rate > 10% of put_rate →
cache too small // hit_rate < 0.8 → key distribution too wide // load_time > 50ms avg → DB
query too slow // Redis monitoring // redis-cli INFO stats: // keyspace_hits, keyspace_misses
→ hitRate = hits/(hits+misses) // evicted_keys → eviction rate // redis-cli INFO memory: //
used_memory, mem_fragmentation_ratio // fragmentation > 1.5 → active defragmentation //
Consistency monitoring: // Compare DB version vs cached version for sampled keys // Alert:
staleness > acceptable_window
```

Interview Questions & Answers

Q1. What is cache eviction and how does it differ from expiration?

Eviction: entry removed because cache is full — capacity-based. Eviction policy selects the victim. Expiration: entry removed because it exceeded TTL — time-based. An expired entry may not be removed immediately (lazy expiration). Both result in a cache miss on next access.

Q2. Explain the LRU eviction algorithm.

Least Recently Used: maintains access order; evicts the entry that has not been accessed for the longest time. Implemented as a doubly-linked list + hashmap (O(1) get/put). Handles temporal locality well. Vulnerability: a sequential scan of keys evicts all hot entries — the scan pollution problem.

Q3. What is the LFU eviction algorithm?

Least Frequently Used: maintains access frequency count per entry; evicts the entry with lowest count. Handles frequency locality well — protects hot entries from scan pollution. Weakness: new hot entries start with count=0 and may be evicted before accumulating frequency (cold start problem). Frequency counts must be periodically decayed to adapt to workload changes.

Q4. What is ARC and why is it considered superior to LRU?

Adaptive Replacement Cache: maintains four lists (T1 recent once, T2 recent twice, B1/B2 ghost history). Self-tunes balance parameter p: if B1 ghosts hit often → increase p (favor recency); if B2 hits → decrease p (favor frequency). Adapts dynamically to workload, outperforming LRU for both scan-heavy and frequency-heavy patterns. Patent prevents open-source use.

Q5. What is W-TinyLFU and why does Caffeine use it?

Window-TinyLFU: combines a Window Cache (1% capacity, new arrivals), Protected Segment (80%, LFU frequent), and Probation Segment (20%, LFU candidates). Admission gate uses Count-Min Sketch frequency filter — new entry admitted only if its estimated frequency exceeds the victim's from Probation. Achieves near-optimal OPT hit rate. Patent-free, so usable in Caffeine.

Q6. What is a Count-Min Sketch?

A probabilistic data structure for frequency estimation using d hash functions and $d \times w$ counter arrays. `increment(key)`: add 1 to d cells. `estimate(key)`: return minimum of d cells. $O(1)$ update and query. Sub-linear memory (not one counter per key). May overcount (false positive frequency) but never undercount. Used by Caffeine's TinyLFU admission filter.

Q7. What is the difference between `expireAfterWrite` and `expireAfterAccess` in Caffeine?

`expireAfterWrite`: TTL measured from creation/last-write time. Guarantees data freshness regardless of access frequency. `expireAfterAccess`: TTL measured from last access. Hot entries stay indefinitely; cold entries expire. Use `expireAfterWrite` for data freshness guarantees; `expireAfterAccess` for memory management of a "hot" working set.

Q8. What is `refreshAfterWrite` and how does it prevent latency spikes?

`refreshAfterWrite(duration)`: after the soft TTL, the next access returns the stale value immediately AND triggers an asynchronous reload via `CacheLoader`. No blocking wait. The fresh value replaces stale when async load completes. Without `refreshAfterWrite`, `expireAfterWrite` causes blocking reloads on TTL expiry — noticeable latency spikes for hot keys.

Q9. What are Redis eviction policies?

`noeviction` (error on OOM), `allkeys-lru` (evict LRU from all keys), `allkeys-lfu` (evict LFU from all), `allkeys-random` (random), `volatile-lru/lfu/random` (only keys with TTL), `volatile-ttl` (soonest-expiring). For cache use: `allkeys-lfu` is recommended for skewed access; `allkeys-lru` for general use. Never use `noeviction` for a cache.

Q10. What does Redis `maxmemory-samples` do?

Redis LRU and LFU are approximate algorithms. `maxmemory-samples N` controls how many keys are sampled per eviction decision. Higher samples = more accurate LRU/LFU approximation but more CPU overhead. Default is 5. Set to 10 for better accuracy. Redis 4+ also has active defragmentation for memory.

Q11. What is cache consistency and why is it difficult?

Cache consistency = ensuring cached data reflects the current ground truth in the DB. Difficult because: multiple cache instances in a cluster diverge, DB updates by external systems bypass cache, concurrent write + evict + read races, distributed systems have no global clock. Solutions: TTL, event-driven invalidation, versioned keys, CDC.

Q12. What is the cache invalidation problem?

Phil Karlton's quote: 'There are only two hard things in Computer Science: cache invalidation and naming things.' Invalidation is hard because: DB may be updated by multiple services, cache may be on multiple nodes, invalidation event may be lost (network failure), and race conditions between write + evict + read can re-populate the cache with stale data.

Q13. What is the delete-then-evict vs evict-then-delete order for cache invalidation?

Evict-then-delete (recommended): remove from cache first, then write to DB. Risk: if DB write fails, cache was evicted unnecessarily (minor — next read reloads). Delete-then-evict: write DB first, then evict cache. Risk: concurrent reader between DB write and cache eviction loads stale data into cache.

Delete-first-then-evict-with-delay (double-delete) handles race.

Q14. What is the double-delete cache invalidation pattern?

On write: 1) delete DB-row-version or update DB, 2) evict cache entry, 3) sleep 500ms, 4) evict cache entry again. The second delete handles the race: if a reader loaded stale data between step 1 and step 2 (from replica lag), the second evict clears that stale entry. Common in MySQL + Redis deployments to handle replica lag.

Q15. What is Change Data Capture (CDC) and how is it used for cache invalidation?

CDC reads the database transaction log (PostgreSQL WAL, MySQL binlog) and emits change events (INSERT, UPDATE, DELETE) to a message broker. Debezium is the most popular Java CDC tool. Cache invalidation consumer subscribes to change events and evicts affected entries. Most reliable: catches ALL DB changes, including direct SQL, migrations, batch jobs.

Q16. What is the ghost list in ARC and W-TinyLFU?

Ghost entries store only the key (hash), not the value, of recently evicted entries. In ARC: B1 and B2 ghost lists track eviction history to tune parameter p . In W-TinyLFU (Caffeine): ghost entries in the frequency sketch maintain frequency counts for recently evicted keys — so if an evicted key returns quickly, it re-enters with its earned frequency and is not evicted immediately again.

Q17. What is the cache staleness window?

The maximum time a cached entry can differ from the DB value. With TTL-based invalidation, staleness window = TTL. With event-driven invalidation, staleness window = event propagation latency (typically < 100ms for Kafka). With CDC: ~50-200ms. With versioned keys: 0 (no staleness possible).

Q18. What are weak references in Java cache implementations?

Caffeine's `weakKeys()` and `weakValues()` store cache entries as `WeakReferences` — eligible for GC when no strong references remain. `softValues()` uses `SoftReferences` — GC'd under memory pressure. Useful for image/session caches where GC can reclaim memory as needed. Hit rate may be lower under memory pressure as GC evicts entries unpredictably.

Q19. What is the SLRU (Segmented LRU) algorithm?

SLRU divides cache into two segments: Protected (hot) and Probation (warm). On first access, entry goes to Probation. On re-access from Probation, promoted to Protected. Eviction from Probation first. If Protected overflows, demoted to Probation. Simpler than ARC but effective for workloads with clear hot/cold distinction. Used as a component in W-TinyLFU.

Q20. How do you implement per-entry TTL in Caffeine?

Use `Expiry` interface with `expireAfterCreate()`, `expireAfterUpdate()`, `expireAfterRead()` methods. Each returns nanoseconds from the respective event. This allows different TTLs per entry based on the value content (e.g., VIP products cached 1 hour, regular products 10 minutes).

Q21. What is scan resistance and why does W-TinyLFU handle it better than LRU?

Scan: sequential access to many unique keys floods an LRU cache, evicting frequently-accessed (hot) entries replaced by scan entries accessed only once. W-TinyLFU's TinyLFU frequency filter admits new entries only if their frequency (from Count-Min Sketch) exceeds the victim's frequency. Scan entries (frequency=0) are rejected, protecting hot entries.

Q22. What is cache warming best practice?

1) Load top-N by access frequency from analytics on startup. 2) Use async warm-up (`ApplicationReadyEvent`) to avoid delaying app start. 3) Batch load with Redis pipeline. 4) Stagger warmup across pods to avoid

overwhelming DB. 5) Monitor cache hit rate after deployment — warm cache should reach > 80% hit rate within minutes.

Q23. What is the difference between strong and eventual cache consistency?

Strong: every read returns the most recent write; cache and DB are always in sync. Requires write-through + cache lock or atomic compare-and-swap. Higher latency. Eventual: cache may lag DB by up to one TTL; subsequent reads converge to DB value. Lower latency, higher availability. Most production caches use eventual consistency.

Q24. What is the fence-based consistency pattern?

Use a monotonically increasing version/fence token with cached values. On read, compare token to DB version. If DB is ahead, evict and reload. Used by Redisson FencedLock: lock holder includes fence token in storage operation; storage rejects writes with lower token. Prevents split-brain: delayed write from paused process cannot overwrite newer data.

Q25. What is the memoization anti-pattern in caching?

Memoization = function-level result caching. Anti-pattern: memoizing functions that produce large or highly variable results (e.g., complex report queries) fills cache and reduces hit rate for simpler hot entries. Also: memoizing within a request (method-level) but not invalidating across requests — stale data persists until the bean is restarted. Limit memoization scope.

Q26. Explain the write-invalidate vs write-update protocol.

Write-invalidate: on write, evict cached entry (invalidate). Next read causes a miss and loads fresh data. Simpler, no risk of stale read. Write-update: on write, update the cached entry with new value (@CachePut). No miss on subsequent reads. Risk: update fails (Redis write error) while DB write succeeds — cache and DB diverge. Write-invalidate is safer; write-update is faster.

Q27. What is the Tombstone pattern for cache invalidation?

After deleting a DB row, write a tombstone marker (e.g., product:42 = DELETED) to cache with short TTL (e.g., 5 seconds). Prevents stale loads: if another thread reads between DB delete and cache eviction, it sees the tombstone and returns null instead of stale value. Tombstone expires, future reads confirm absence from DB.

Q28. How does the Redis LFU algorithm work?

Redis approximates LFU using a logarithmic counter per key (0-255, stored in the key's idle-time field). Counter incremented on access with probability: $p = 1/(\text{old_value} * \text{lfu_log_factor} + 1)$. Counter decremented every lfu_decay_time minutes of no access. This aging prevents old-hot items from monopolizing cache forever. lfu_log_factor (default 10) controls counter saturation speed.

Q29. What is a consistent-hash based eviction across Redis Cluster?

In Redis Cluster, each node independently manages its own memory and applies eviction per-node. There is no global LRU/LFU across the cluster. Eviction on one node does not know about access patterns on other nodes. This means globally cold keys on heavily-loaded nodes may be evicted while globally hot keys on lightly-loaded nodes survive. Solution: rebalance cluster slots or use per-key TTL for fine-grained control.

Q30. How do you detect and fix a low cache hit rate?

Diagnose: 1) Check eviction rate — if high, cache too small; increase maxSize. 2) Check key distribution — if highly diverse (many unique params), consider more selective caching. 3) Check TTL — if too short relative to request frequency, entries expire before reuse. 4) Check for stampede — if hit rate recovers then drops cyclically, stampede on cache expiry. Fix: increase cache size, extend TTL, add bloom filter, use refresh-ahead.

Q31. What is the poison-entry problem in caching?

A cache entry is populated with an error result (e.g., null, empty list, exception). On subsequent reads, the error is served from cache — bypassing the DB even though the error was transient. Solutions: never cache errors (unless vs cache, catch exceptions before caching), set short TTL for null/error results, use `@Cacheable unless='#result==null'`.

Q32. What monitoring signals indicate a healthy cache?

Hit rate > 85% (L1 Caffeine), > 70% (L2 Redis). Eviction rate < 5% of put rate. Miss latency dominated by DB query, not lock contention. Memory usage stable (not growing). p99 cache get < 1ms (Caffeine), < 5ms (Redis). No connection timeouts. Staleness alarms < 1% of entries beyond SLA window. Redis fragmentation ratio < 1.5.